

UNIT-II Relational Model

①

* Introduction to the relational model:-

What is Relational Model:-

The relational model represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real world entity or relationship.

The table name and column names are helpful to interpret the meaning of values in each row. The data are represented as a set of relations. In the relational model, data are stored as tables.

However, the physical storage of the data is independent of the way the data are logically organized.

Some popular Relational database management systems are:

- DB2 and Informix Dynamic Server - IBM
- Oracle and RDB - Oracle
- SQL server and Access - Microsoft

Relational Model Concepts:-

1. Attribute:- Each column in a table. Attributes are the properties which define a relation. ex: Stu-Rollno, Name etc.
2. Tables:- In the relational model the relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.

3. Tuple:- It is nothing but a single row of a table, which contains a single record. tuple also called record.

4. Relation schema:- A relation schema represents the name of the relation with its attributes.

5. Degree:- The total number of attributes which in the relation is called the degree of the relation. The degree, also called arity, of a relation is the number of fields.

6. Cardinality:- Total number of rows present in the table.

7. Column:- The column represents the set of values for a specific attribute.

8. Relation instance:- Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.

9. Relation key:- Every row has one, two or multiple attributes, which is called relation key.

10) Attribute domain:- Every attribute has some pre-defined values and scope which is known as attribute domain.

Primary key		Domain Ex:- NOT NULL
customer ID	customer Name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Enactive

Tuple OR Row
Total # of row is
cardinality.

Column OR Attributes
Total # of column is Degree

A relational database is a collection of relations with distinct relation names. A relational database schema is the collection of schemas for the relations in the database.

Ex:- A university database with relations called Students, Faculty, Courses, Rooms, Enrolled, Teaches, and Meets. In an Instance of a relational database is a collection of relation instances, one per relation schema in the database schema; Of course, each relation instance must satisfy the domain constraints in its schema.

Advantages:-

1. Simplicity:- A relational data model is simpler than the hierarchical and network model.
2. Structural Independence:- The relational database is only concerned with data and not with a structure. This can improve the performance of the model.
3. Easy to use:- The relational model is easy as tables consisting of rows and columns is quite natural and simple to understand.
4. Query Capability:- It makes possible for a high-level query language like SQL to avoid complex database navigation.
5. Data Independence:- The structure of a database can be changed without having to change any application.
6. Scalable:- Regarding a number of records, or rows, and the number of fields, a database should be enlarged to enhance its usability.

Disadvantages:-

1. Few relational databases have limits on field lengths which can't be exceeded.
2. Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
3. complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.

* Integrity Constraints Over Relations:-

An integrity constraint (IC) is a condition specified on a database schema and restricts the data that can be stored in an instance of the database. If a database instance satisfies all the integrity constraints specified on the database schema, it is a legal instance. A DBMS enforces integrity constraints, in that it permits only legal instances to be stored in the database.

Integrity constraints ensure that changes (update, deletion, insertion) made to the database by authorized users do not result in a loss of data consistency.

Integrity constraints are specified and enforced at different times:

- 1) when the DBA or End user defines a database schema, he or she specifies the ECS must hold on any instance of this database
- 2) when a database application is run, the DBMS checks for violations and disallows changes to the data that violate the specified ECS. When integrity constraints are checked relative to the statement that causes the change in the data and the transaction that it is part of.

Constraints enforce limits to the data or type of data that can be inserted / updated / deleted from a table. The whole purpose of constraints is to maintain the data integrity during an update / delete / insert into a table.

There are 6 different types of constraints:

- 1) Domain constraint
- 2) Tuple uniqueness constraint
- 3) Key constraint
- 4) Entity Integrity constraint
- 5) Referential Integrity constraint
- 6) General constraints

The mainly discuss with 3 constraints:

- 1) Key constraint
- 2) Referential Integrity constraint
- 3) General constraints

1) key constraint:- key constraint specifies that in any relation

→ All the values of primary key must be unique

→ The values of primary key must not be null,

Ex:-

Stu-ID	Name	Age
S001	Akshay	20
S001	Abhishek	21
S003	Shashank	20
S004	Rahul	20

∴ This relation does not satisfy the key constraint as here all the values of primary key are not unique.

Unique key:-

- 1) No duplicate values
- 2) but null values.

Ex:- Create table students (sid char(20), name char(30), login char(20), age integer, gpa Real,

1)

Unique (name, age), constraint studentskey

primary key (sid))

∴ This table Ex:- 2)

Sid	Sname	branch
101	Sai	CSE
102	Shiva	CSE
-	Sri	CSE

A key constraint is a statement that a certain minimal subset of the fields of a relation is a unique identifier for a tuple. A set of fields that uniquely identifies a tuple according to a key constraint is called a candidate keys for the relation.

→ Two distinct tuples in a legal instance (an instance that satisfies all DCs including the key constraint) cannot have identical values in all the fields of a key.

→ No subset of the set of fields in a key is a unique identifier for a tuple.

2) Referential Integrity constraint: - This constraint is enforced when a foreign key references the primary key of a relation.

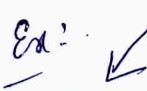
It specifies that all the values taken by the foreign key must either be available in the relation of the primary key or be null.

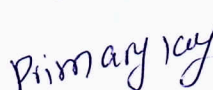
There are two different types of keys:

- 1) primary key
- 2) foreign key constraint

1) primary key constraint: - The combination of unique and NOT NULL.

- 1) No duplicate values.
- 2) No null values.

Ex: 

Primary key 

Sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@cs	19	3.2
53650	Madayan	madayan@music	11	3.8

Students table.

Create table Students (sid int primary key, name char(15),
login char(20), age number(2), gpa real);

2) foreign key constraint: - The information stored in a relation is linked to the information stored in another relation. If one of the relation is modified, the other must be checked, and perhaps modified, to keep the data consistent. The most common BC involving two relations is a foreign key constraint.

(1)

Ex:

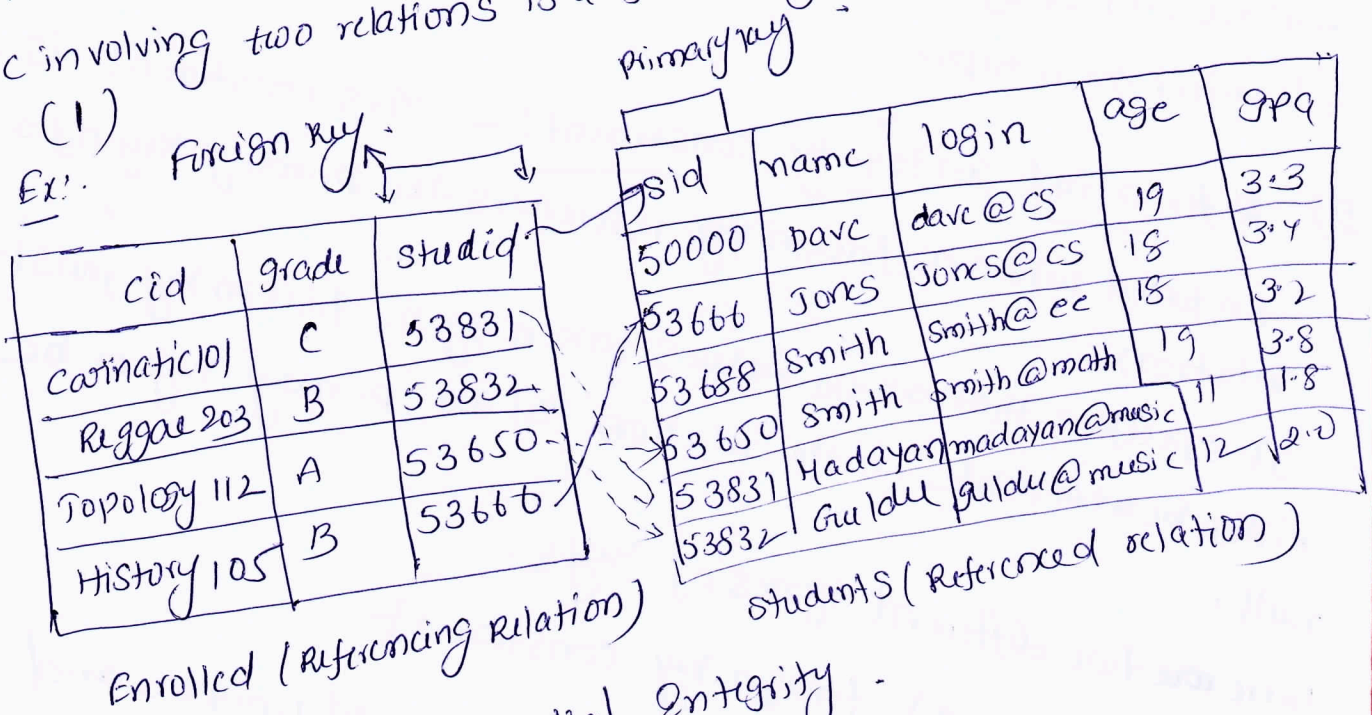


Fig.: Referential Integrity.

Ex: i) Students table creation:

Create table Students (sid int primary key, name char(15),
login char(20), age number(2), gpa real);

ii) Enrolled table creation: using primary key and foreign key:

Create table Enrolled (studid char(20), cid char(20), grade
char(10), primary key (studid, cid),
foreign key (studid) references Students);

Ex:-(2)

primary key

Dept-id	Dname
10	CSE
20	ECE
30	CIVIL

primary key

Emp-id	E-name	Sal	dept-id
1	A	10000	10
2	B	20000	20
3	C	30000	30

foreign key

To create Dept table:-

create table Dept (dept-id number(10) primary key,
Dname char(10));

To create Employee table:-

create table Employee (Emp-id number(10) primary key,
E-name char(10), Sal real, foreign key(
dept-id) references Dept(dept-id));

NULL:- The use of null in a field of a tuple means that value in that field is either unknown or not applicable. The appearance of null in a foreign key field does not violate the foreign key constraint. However, null values are not allowed to appear in a primary key field (because the primary key fields are used to identify a tuple uniquely).

(01)

Sid	Sname	age
101	A	16
102	B	17
-	C	18

null value

Sid	Sname	age	phoneno
1	A	10	1234567
2	B	20	145678
3	C	30	Null

3) General Constraints:-

current relational database systems support such general constraints in the form of table constraints and assertions.

Table constraints:- Table constraints are associated with a single table and checked whenever that table is modified.

Assertions:- Assertions involve several tables and are checked whenever any of these tables is modified.

Both table constraints and assertions can use the full power of SQL queries to specify the desired restrictions.

* Operations in relational model:-

Four basic update operations performed on relational database model are insert, update, delete and select.

- insert is used to insert data into the relation
- delete is used to delete tuples from the table.
- Modify allows you to change the values of some attributes in existing tuples
- Select allows you to choose a specific range of data.

Whenever one of these operations are applied, integrity constraints specified on the relational database schema must never be violated.

Insert operation:- The insert operation gives values of the attribute for a new tuple which should be inserted into a relation.

Customer ID	customer name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive



customer ID	customer name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive
4	Alibaba	Active

Update operation:- We can see that in the below-given relation table customer name = 'Apple' is updated from Inactive to Active

Cust_ID	cust_name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive
4	Alibaba	Active



cust_ID	cust_name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Active
4	Alibaba	Active

Delete operation:- To specify deletion, a condition on the attributes of the relation selects the tuple to be deleted.

cust_ID	cust_name	Status
1	Google	Active
2	Amazon	Active
3	Apple	Active
4	Alibaba	Active



cust_ID	cust_name	Status
1	Google	Active
2	Amazon	Active
3	Alibaba	Active

In the above-given example, cust_name = "Apple" is deleted from the table

The delete operation could violate referential integrity if the tuple which is deleted is referenced by foreign keys from other tuples in the same database.

Select operation:-

cust_ID	cust_Name	status
1	Google	Active
2	Amazon	Active
4	Alibaba	Active

Select

cust_ID	cust_Name	status
2	Amazon	Active

In the above given example, cust_Name = "Amazon" is selected.

* Enforcing Integrity Constraints:-

Integrity constraints are specified when a relation is created and enforced when a relation is modified. The impact of domain, primary key, and unique constraints is straightforward; if an insert, delete, or update command causes a violation, it is rejected. Every potential integrity constraint violation is generally checked at the end of each SQL statement execution, although it can be deferred until the end of the transaction executing the statement.

Field names

(Tuples, Records, Rows)

sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53650	Smith	smith@math	19	3.8
53831	Madayan	madayan@music	11	1.8
53832	Guldu	guldu@music	12	2.0

Fig: An instance S_1 of the students relation

The following insertion violates the primary key constraint because there is already a tuple with the sid 53688, and it will be rejected by the DBMS:

```
INSERT
  INTO students (sid, name, login, age, gpa)
VALUES (53688, 'Mike', 'mike@ee', 17, 3.4)
```

The following insertion violates the constraint that the primary key cannot contain null:

```
INSERT
  INTO students (sid, name, login, age, gpa)
VALUES (null, 'Mike', 'mike@ee', 17, 3.4)
```

Deletion does not cause a violation of domain, primary key or unique constraints. However, an update can cause violations, similar to an insertion:

```
UPDATE students S
  SET S.sid = 50000
  WHERE S.sid = 53688;
```

This update violates the primary key constraint because there is already a tuple with sid 50000.

We discuss the referential integrity enforcement steps taken by the DBMS in terms of our Enrolled and Students table, with the foreign key constraint that Enrolled.sid is a reference to Students.

With the references of Referential Integrity figure:

Deletion of Enrolled tuples do not violate referential integrity, but insertion of Enrolled tuples could. The following insertion is illegal because there is no Students tuple with sid 51111.

```

INSERT
INTO Enrolled (cid, grade, studied)
VALUES ('Hindiloi', 'B', 'S1111');

```

Consider students and enrolled tables:

→ sid in Enrolled is a foreign key that references the student table

→ what should be done if an enrolled tuple with a nonexistent student id is inserted?

- Reject it.

→ what should be done if a student, s tuple is deleted?

- Also delete all enrolled tuples that refer to it.
- Disallow deletion of a student's tuple that is referred to.
- Set sid in enrolled tuples that refer to it to a default sid.

(. In SQL also: Set sid in enrolled tuples that refer to it to a special value null, denoting 'unknown' or 'inapplicable')

→ Similar if primary key of students tuple is updated.

Specifying behavior on referential integrity violation:-

→ Behavior specified at table create

- No action (Reject action that violates constraint)
- Update referring table (update foreign key to the new value)
- Set to NULL (set all foreign keys to a NULL)
- Set to a default (set all foreign keys to a default value)

Create table Enrolled (sid char(20), cid char(20), grade char(2),
 primary key (sid, cid), foreign key (sid) References Students ON
 DELETE CASCADE ON UPDATE SET DEFAULT)

* Querying Relational Data:-

A relational database query (query, forshort) is a question about the data, and the answer consists of a new relation containing the result. A query language is a specialized language for writing queries.

SQL is the most popular commercial query language for a relational DBMS.

Ex:-

Sid	name	login	age	gpa
50000	Dave	dave@cs	19	3.3
53666	Jones	Jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2
53650	Smith	Smith@math	19	3.8
53831	Madayan	madayan@music	11	1.8
53832	Guldu	Guldu@music	12	2.0

Fig:- An instance S_1 of the Students Relation.

Query:- 1) ~~For~~ we can retrieve rows corresponding to students who are younger than 18 with the following SQL query:-

```
SELECT *
FROM Students S
WHERE S.age < 18;
```

The Symbol '*' means that we retain all fields of selected tuples in the result. S as a variable that takes on the value of each tuple in Students, one tuple after the other. The condition $S.age < 18$ in the WHERE clause specifies that we want to select only tuples in which the age field has a value less than 18.

Sid	name	login	age	gpa
53831	Madayan	Madayan@music	11	1.8
53832	Guldu	Guldu@music	12	2.0

Fig: Students With age < 18 on Instance S1

The condition $S.age < 18$ involves an arithmetic comparison of an age value with an integer and is permissible because the domain of age is the set of integers.

Query 2: - We can compute the names and logins of students who are younger than 18.

```
SELECT S.name, S.login
FROM Students S
WHERE S.age < 18;
```

name	login
Madayan	madayan@music
Guldu	guldu@music

Fig: Names and Logins of Students Under 18

Query 3: - We can also combine information in the Students and Enrolled relations. If we want to obtain the names of all students who obtained an A and the id of the course in which they got an A.

```
SELECT S.name, E.cid
FROM Students S, Enrolled E
WHERE S.sid = E.stuid AND E.grade = 'A';
```

This query can be understood as follows: "If there is a students tuple S and an Enrolled tuple E such that $S.sid = E.studid$ (so that S describes the student who is Enrolled in E) and $E.grade = 'A'$, then print the student's name and the courseid".

* Logical Database Design : ER To Relational

The ER model is convenient for representing an initial, high-level database design.

1) Entity Sets to Tables: - Each attribute of the Entity set becomes an attribute of the table. The domain of each attribute and the (primary) key of an Entity set.

Ex! - consider the Students entity set with attributes sid , $sname$ and age .

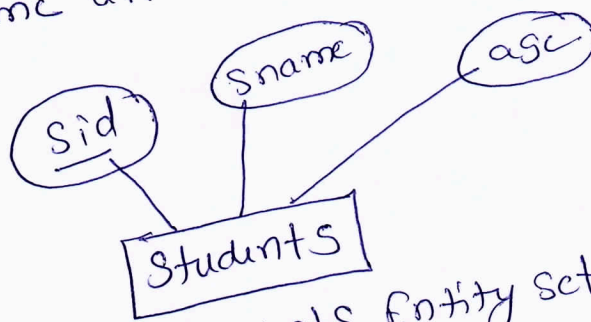


Fig: Students Entity set.

Table: -

sid	sname	age
101	A	20
102	B	21
103	C	22

Fig: An instance of the Students Entity set

Table creation: -

Create table Students (sid int, $sname$ char(10), age number(10), primary key (sid));

2) Relationship sets (without constraints) to Tables:-

A relationship set, like an entity set, is mapped to a relational ~~model~~ in the relational model.

The attributes of the relation include:

→ The primary key attributes of each participating entity set, as foreign key fields -

→ The descriptive attributes of the relationship set.

Ex: consider the Works - In relationship set.
Each Department has offices in several locations and we want to record the locations at which each employee works.

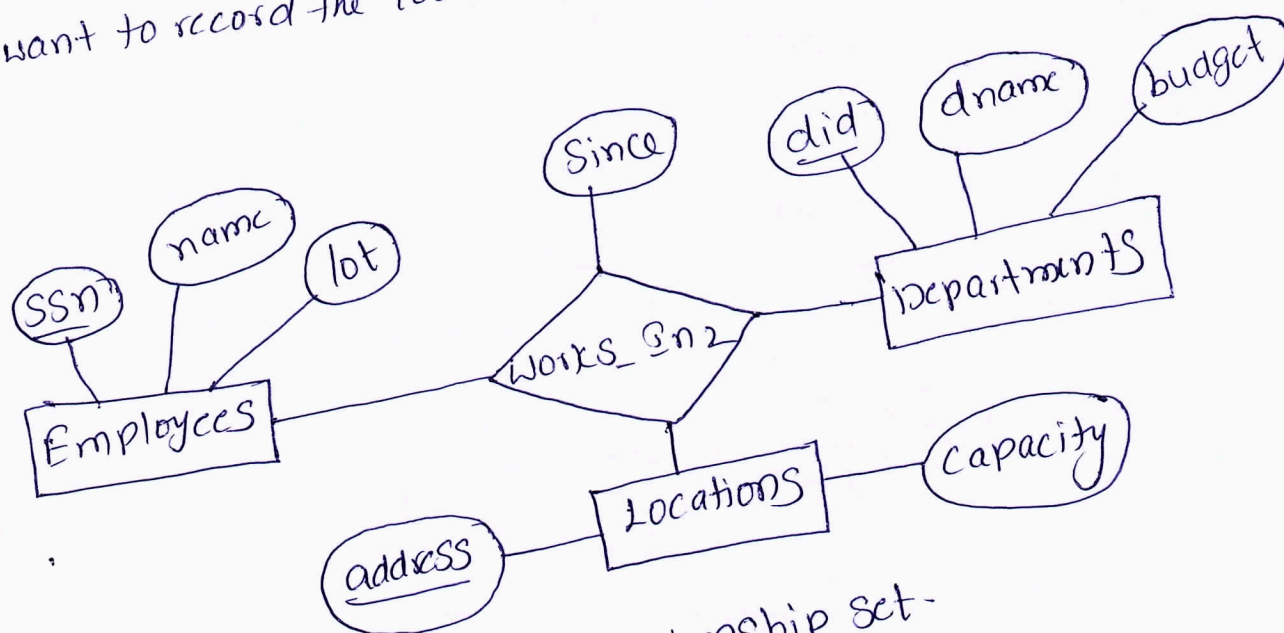


Fig: A Ternary Relationship Set-

Create table Works_In (SSN char(11),
did integer,
address char(20),
since date,

primary key (SSN, did, address),

foreign key (SSN) references Employees,

foreign key (address) references Locations,

foreign key (did) references Departments);

address, did and SSN fields cannot take on null values. Because these fields are part of the primary key for works_Sn2, a NOT NULL constraint is implicit for each of these fields.

Consider the Reports-To Relationship set

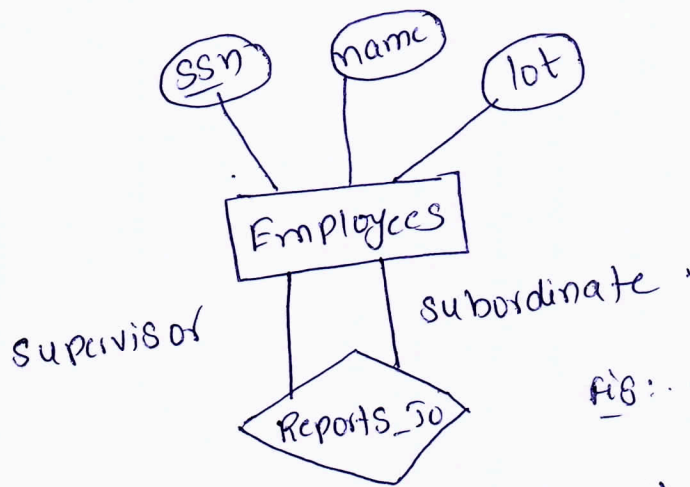


Fig: The Reports-To Relationship set

The role indicators supervisor and subordinate are used to create meaningful field names in the Create statement for the Reports-To table:

```

Create table Reports-To (supervisor_Ssn char(11),
subordinate_Ssn char(11), primary key(supervisor_Ssn,
subordinate_Ssn), foreign key(supervisor_Ssn) references
Employee (ssn), foreign key(subordinate_Ssn) references
Employees (ssn));
  
```

Translating Relationship sets with key constraints: -

If a relationship set involves n entity sets and some m of them are linked via arrows in the ER diagram, the key for any one of these m entity sets constitutes a key for the relation to which the relationship set is mapped. Hence we have m candidate keys, and one of these should be designated as the primary key.

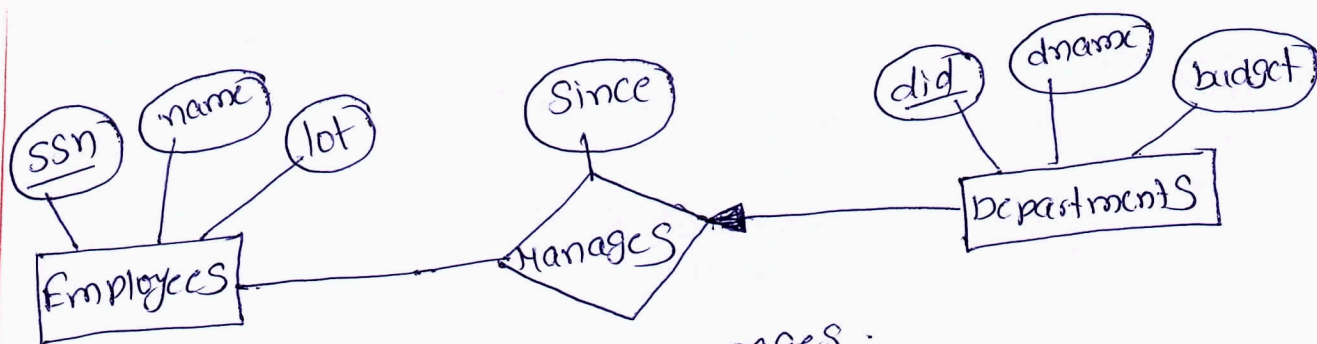


Fig: key constraint on Manages.

The table corresponding to Manages has the attributes SSN, did, since. Each department has at most one manager, no two tuples can have the same did value but differ on the SSN value.

Create table Manages (SSN char(11), did integer, since date, primary key (did), foreign key (SSN) References Employees, foreign key (did) References Departments);

A second approach to translating a relationship set with key constraints is often superior because it avoids creating a distinct table for the relationship set.

Ex:- A department has at most one manager, we can add the key field of the employees tuple denoting the manager and the since attribute to the department tuple.

This approach eliminates the need for a separate Manages relation, and queries asking for a department's manager can be answered without combining information from two tables.

The following SQL statement, defining a dept_mgr relation that captures the information in both departments and Manages.

The second approach to translating relationship sets with key constraints:

Create table Dept_Mgr (did Integer,
 dname char(20),
 budget Real,
 ssn char(11),
 since Date,
 primary key (did),
 foreign key (ssn) references employees);

SSN can take on null values.

Translating Relationship sets with participation constraints:-

Every department is required to have a manager, due to the participation constraint, and at most one manager, due to the key constraint.

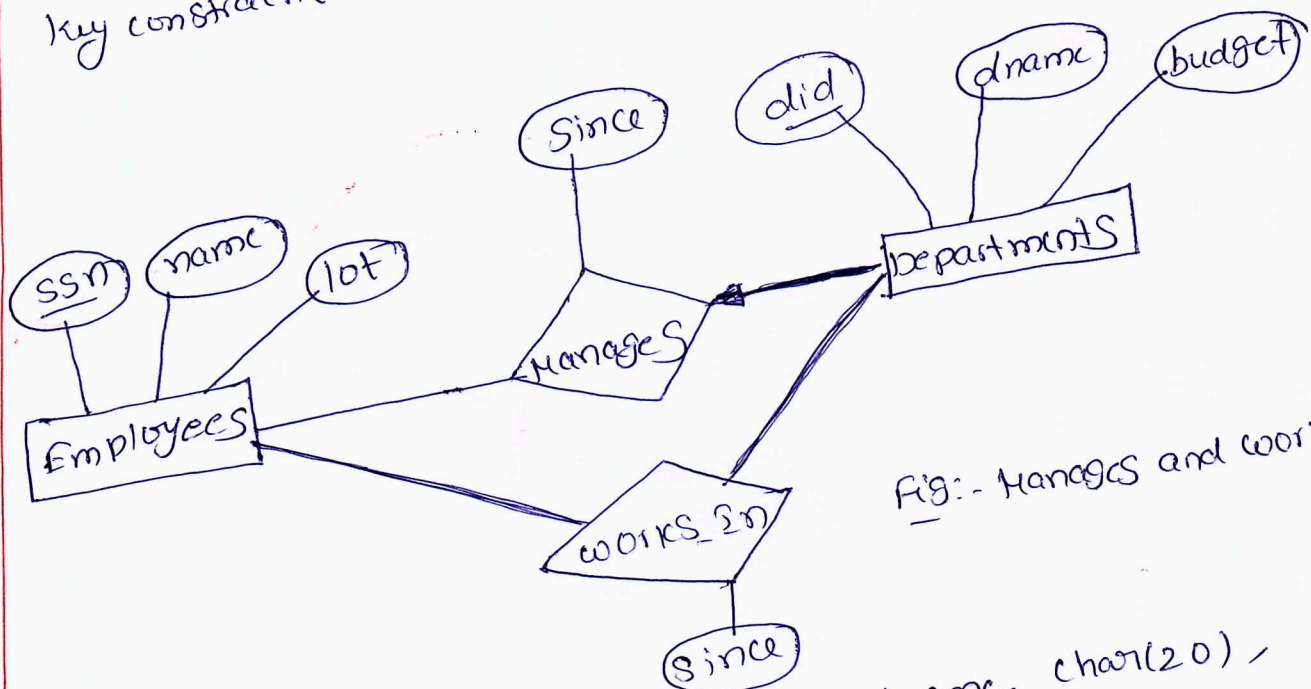


Fig:- Manages and works_in

Create table Dept_Mgr (did Integer, dname char(20),
 budget Real, ssn char(11) NOT NULL, since Date,
 primary key (did), foreign key (ssn) references Employee
 ON DELETE NO ACTION);

SSN cannot take on null values, each tuple of Dept_Hgt identifies a tuple in Employees. The NO ACTION specification, which is the default and need not be explicitly specified, ensures that an Employees tuple cannot be deleted while it is pointed to by a Dept_Hgt tuple. If we wish to delete such an Employees tuple, we must first change the Dept_Hgt tuple to have a new Employee as manager.

Translating weak Entity sets:-

A weak Entity set always participates in a one-to-many binary relationship and has a key constraint and total participation.

Consider the dependents weak entity set, with partial key pname. A dependents entity can be identified uniquely only if we take the key of the owning Employees entity and the pname of the dependents entity, and the dependents entity must be deleted if the owning Employees entity is deleted.

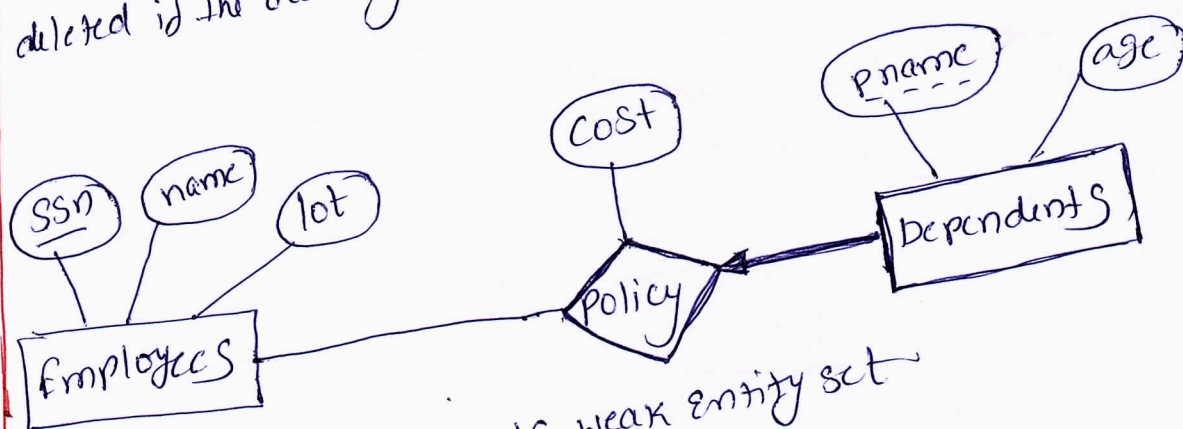


Fig: The dependents weak entity set

Create table dep-policy (pname char(20), age integer, cost real, SSN char(11), primary key (pname, SSN), foreign key (SSN) references EMPLOYEES ON DELETE CASCADE)

The CASCADE option ensures that information about an employee's policy and dependents is deleted if the corresponding employee's tuple is deleted.

Translating class Hierarchies:-

We present the two basic approaches to handling ISA hierarchies

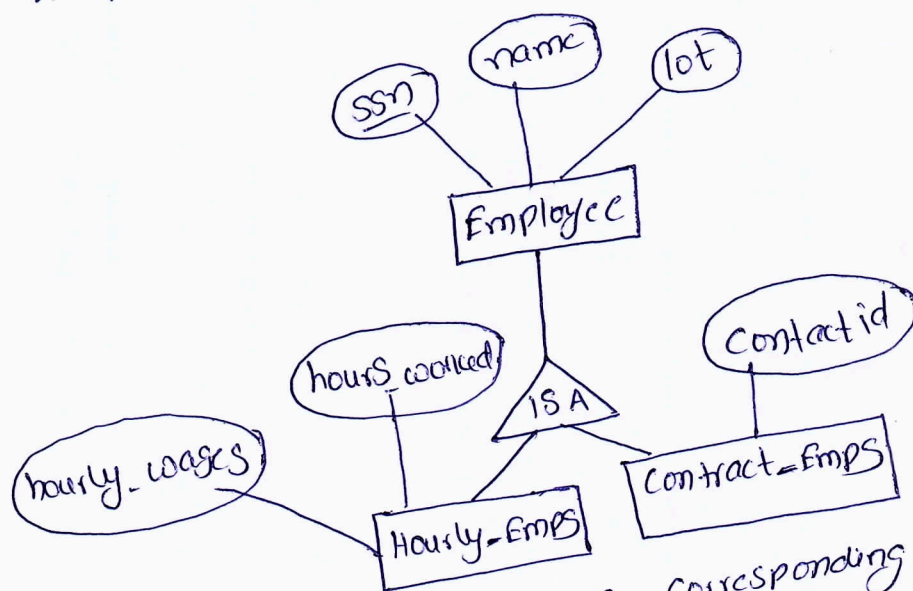


Fig: class hierarchy.

We can create just two relations, corresponding to Hourly Emps and Contract_Emps. The relation for Hourly_Emps includes all the attributes of Hourly_Emps as well as all the attributes of Employees. → All employees and do not care about the attributes specific to the subclasses are handled easily using the Employees relation. → who are neither hourly employees, nor contract employees, since there is no way to store such employee.

Translating ER Diagrams with aggregation:-

1. we have to record the descriptive attributes (in our example, since) of the sponsors relationship.
2. Not every sponsorship has a monitor, and thus some(pid, did) pairs in the sponsors relation may not appear in the monitors relation.

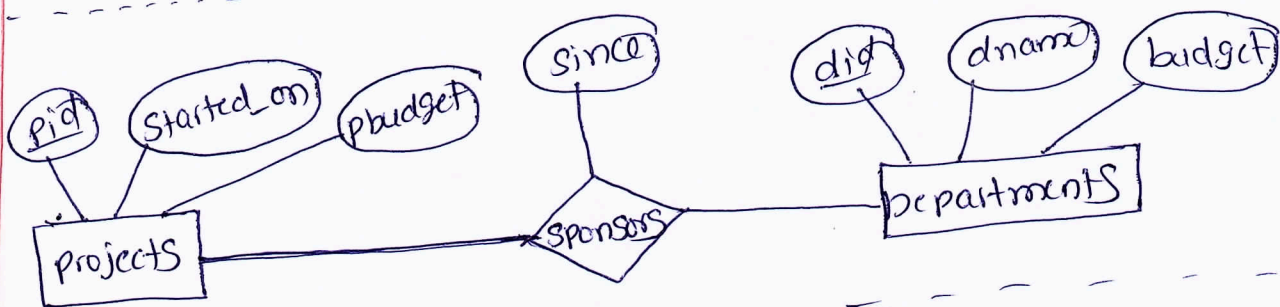
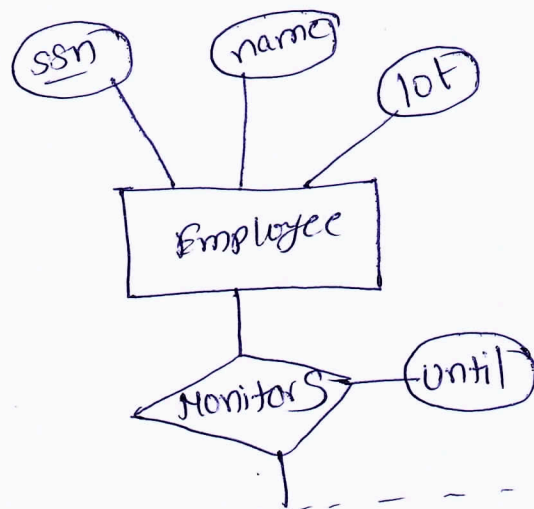
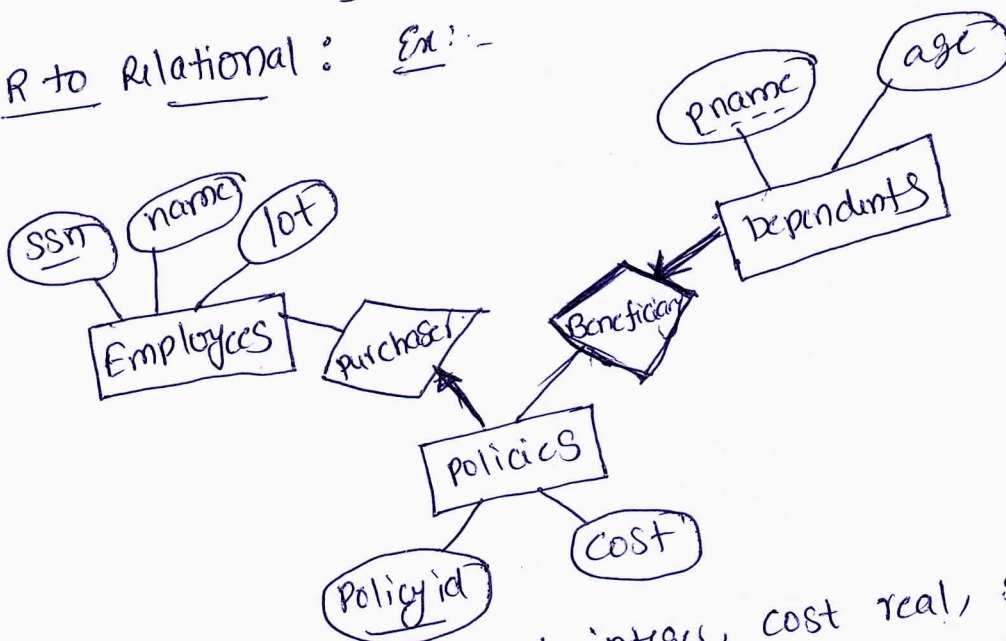


Fig:- Aggregation

ER to Relational: Ex:-



- create table policies (policyid integer, cost real, SSN char(11) not null, primary key (policyid), foreign key (SSN) references Employees ON DELETE CASCADE);
- create table dependents (pname char(20), age integer, policyid int, primary key (pname, policyid), foreign key (policyid) references policies ON DELETE CASCADE);

* Introduction to views: -

A view in SQL is a logical subset of data from one or more tables.

view is used to restrict data access.

→ views in SQL are considered as a virtual table. A view also

contains rows and columns.

→ To create the view, we can select the fields from one or more tables present in the database.

→ A view can either have specific rows based on certain condition or all the rows of a table.

Ex:- sample table:-

student_detail:-

stu_ID	Name	Address
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

student_marks:-

stud ID	Name	Marks	Age
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1) Creating view:- A view can be created using the create view statement. We can create a view from a single table or multiple tables.

Syn:-

```
Create view view_name AS
SELECT column1, column2 ...
From table_name
WHERE condition;
```

2) Creating view from a single table:-

We can create a view named detailsview from the table student_detail.

Query:- Create view detailsview AS
Select name, Address
from student_detail
where stu_ID < 4;

We can query the view to view the data.

```
Select * from detailsview;
```

O/P:-

Name	Address
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

3) Creating view from multiple tables:-

view from multiple tables can be created by simply include multiple tables in the select statement.

A view is created named Marksview from two tables student_detail and Student_Marks.

Query: Create view MarksvieW AS

SELECT Student_Detail.Name, Student_Detail.Address,
Student_Marks.Marks
FROM Student_Detail, Student_Mark
WHERE Student_Detail.Name = Student_Marks.Name

To display data of view MarksvieW;

SELECT * FROM MarksvieW;

Name	Address	Marks
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74
Alina	Gurgaon	90

4. Deleting view: - A view can be deleted using the DROP view statement.

Syn: DROP VIEW view_name;

Ex: If we want to delete the view MarksvieW.

DROP VIEW MarksvieW;

5. Update a view: - Update command for view is same as for tables.

Syn: UPDATE view_name SET = value WHERE condition;

Types of view:- There are two types of view -

- simple view
- complex view

simple view	complex view
created from one table	created from one or more table
does not contain functions	contain function
Do not contain groups of data	contains groups of data

* Altering / Destroying Tables And views:-

If we decide that we no longer need a base table and want to destroy it (i.e., delete all the rows and remove the table definition information), we can use the DROP TABLE Command.

Ex:- DROP TABLE students RESTRICT destroys the students table unless some view or integrity constraint refers to students; if so, the command fails. If the keyword RESTRICT is replaced by CASCADE, students is dropped and any referencing views or integrity constraints are (recursively) dropped as well; A view can be dropped using the DROP view command, which is just like DROP TABLE.

ALTER TABLE modifies the structure of an existing table.
To add a column called maiden_name to Students.

```
ALTER TABLE Students  
ADD COLUMN maiden_name char(10);
```

The definition of Students is modified to add this column, and all existing rows are padded with null values in this column. ALTER TABLE can also be used to delete columns and add or drop integrity constraints on a table;

Drop: table: drop table <tablename>;
Drop: view: drop view view-name;

